462. Minimum Moves to Equal Array Elements II

Daily Question - November 2, 2026

Problem Statement

Given an integer array nums of size n, return the minimum number of moves required to make all array elements equal.

In one move, you can increment or decrement one element of the array by 1. Test cases are designed so that the answer fits in a 32-bit integer.

Examples

Example 1: nums = [1,2,3]. Output: 2. Explanation: [1,2,3] -> [2,2,3] -> [2,2,2].

Example 2: nums = [1,10,2,9]. Output: 16. Explanation: Sorted array is [1,2,9,10]. Using median 9, the moves are |1-9| + |2-9| + |9-9| + |10-9| = 8 + 7 + 0 + 1 = 16.

Constraints

n == nums.length; 1 <= n <= 100000; -1000000000 <= nums[i] <= 1000000000.

Key Insight

Changing an element x to a target t costs |x - t| moves. Therefore, the total cost for target t is the sum of absolute distances from every element to t. The target that minimizes this sum is the median of the array.

Sort the array and choose nums[n / 2] as the target. For an even-length array, either middle element gives the same minimum total; this solution uses the upper median.

Complete Java Solution

```
import java.util.Arrays;

class Solution {
    public int minMoves2(int[] nums) {
        Arrays.sort(nums);

        int median = nums[nums.length / 2];
        long moves = 0;

        for (int value : nums) {
            moves += Math.abs((long) value - median);
        }

        return (int) moves;
    }
}
```

Correctness

For a sorted array, suppose the target lies between two adjacent elements a and b. Moving the target one step toward the median decreases the distance for at least half of the elements and increases it for at most half. Thus the total absolute-distance cost cannot decrease outside the median region. The median therefore minimizes the sum of absolute differences. Each element contributes exactly its absolute difference from the median, so summing those differences gives the minimum number of moves.

Complexity

Time: O(n log n), dominated by sorting. Auxiliary space: O(log n) for the sorting recursion stack in Java. The final answer is guaranteed to fit in an int.

Implementation Notes

Use long while accumulating moves to prevent intermediate overflow, since values can be as large as 1,000,000,000 in magnitude and n can be 100,000. Cast the final accumulated total to int only after the loop. Arrays.sort modifies nums in place, which is acceptable for this problem.